

Name: Slayde Sequeira

Roll No: 9638

Experiment No 11

Aim: Write queries to sort and aggregate the data in a table using HiveQL

Objective:

The objective of this experiment is to demonstrate how to write queries to sort and aggregate data stored in a Hive table using HiveQL.

Tools and Technologies:

- Hive: A data warehouse infrastructure built on top of Hadoop for providing data summarization, query, and analysis.
- HiveQL: Hive Query Language, which is similar to SQL and used for querying and managing data in Hive.

1. Creating the Table in Hive

The screenshot shows the Hive web interface. At the top, there's a header with the Hive logo, a 'Add a name...' button, and a link to 'Email Survey Opt-Ins, Custom...'. Below the header, a toolbar contains icons for charting, saving, and help. The main area displays a SQL query for creating a table named 'wine_data'. The query is as follows:

```
1 CREATE TABLE wine_data (  
2     fixed_acidity DOUBLE,  
3     volatile_acidity DOUBLE,  
4     citric_acid DOUBLE,  
5     residual_sugar DOUBLE,  
6     chlorides DOUBLE,  
7     free_sulfur_dioxide DOUBLE,  
8     total_sulfur_dioxide DOUBLE,  
9     density DOUBLE,  
10    pH DOUBLE,  
11    sulphates DOUBLE,  
12    alcohol DOUBLE,  
13    quality INT  
14 ) ROW FORMAT DELIMITED  
15 FIELDS TERMINATED BY ','  
16 STORED AS TEXTFILE;  
17
```

Below the query, a status bar indicates '0.37s default' with settings and help icons. A console window at the bottom shows the execution log:

```
FIELDS TERMINATED BY ','  
STORED AS TEXTFILE  
INFO : Starting task [Stage-0:DDL] in serial mode  
INFO : Completed executing command(queryId=hive_20241108171653_ed373156-9532-477e-b561-30ea520141f0); Time taken: 0.031 seconds
```

✓ Success.

Query History

Saved Queries

a minute ago



```
-- Get email survey opt-in values for all customers SELECT c.id, c.fname,  
FROM customers c; -- Select customers for a given shipping ZIP Code SELECT  
customers.id, customers.name FROM customers  
WHERE customers.addresses['shipping'].zip_code = '76710';  
-- Compute total amount per order for all customers SELECT c.id AS customer_id,  
c.name AS customer_name, ords.order_id AS order_id,  
SUM(order_items.price * order_items.qty) AS total_amount FROM customers c  
LATERAL VIEW EXPLODE(c.orders) o AS ords  
LATERAL VIEW EXPLODE(ords.items) i AS order_items  
GROUP BY c.id, c.name, ords.order_id;
```

2. Inserting data into Hive

 Hive

 

Add a name... Email Survey Opt-Ins, Custom...

    

2.26s default  

```
1 INSERT INTO TABLE wine_data VALUES
2   (7.4, 0.7, 0.0, 1.9, 0.076, 11, 34, 0.9978, 3.51, 0.56, 9.4, 5),
3   (7.8, 0.88, 0.0, 2.6, 0.098, 25, 67, 0.9968, 3.2, 0.68, 9.8, 5),
4   (7.8, 0.76, 0.04, 2.3, 0.092, 15, 54, 0.997, 3.26, 0.65, 9.8, 5),
5   (11.2, 0.28, 0.56, 1.9, 0.075, 17, 60, 0.998, 3.16, 0.58, 9.8, 6),
6   (7.4, 0.7, 0.0, 1.9, 0.076, 11, 34, 0.9978, 3.51, 0.56, 9.4, 5),
7   (7.9, 0.6, 0.06, 1.6, 0.069, 15, 59, 0.9964, 3.3, 0.46, 9.4, 6),
8   (7.3, 0.65, 0.0, 1.2, 0.065, 15, 21, 0.9946, 3.39, 0.47, 10.0, 6),
9   (7.8, 0.58, 0.02, 2.0, 0.073, 9, 18, 0.9968, 3.36, 0.57, 9.5, 5),
10  (7.5, 0.5, 0.36, 6.1, 0.071, 17, 102, 0.9978, 3.35, 0.8, 10.5, 5),
11  (6.7, 0.58, 0.08, 1.8, 0.097, 15, 65, 0.9959, 3.28, 0.54, 9.2, 5),
12  (7.1, 0.59, 0.0, 1.9, 0.08, 8, 35, 0.9962, 3.47, 0.53, 9.3, 5),
13  (6.8, 0.54, 0.0, 2.4, 0.092, 12, 38, 0.9966, 3.45, 0.57, 10.1, 6),
14  (7.3, 0.65, 0.0, 1.2, 0.065, 15, 21, 0.9946, 3.39, 0.47, 10.0, 6),
15  (7.8, 0.61, 0.29, 1.6, 0.114, 9, 29, 0.9974, 3.26, 1.56, 9.1, 5),
16  (7.5, 0.31, 0.22, 6.8, 0.081, 35, 60, 0.9964, 3.38, 0.59, 9.9, 5),
17  (10.5, 0.31, 0.56, 6.2, 0.068, 16, 42, 0.9969, 3.24, 0.57, 9.7, 5),
18  (6.3, 0.39, 0.16, 1.4, 0.08, 24, 32, 0.9949, 3.42, 0.58, 10.6, 6),
19  (6.8, 0.67, 0.02, 1.8, 0.08, 7, 21, 0.996, 3.38, 0.59, 9.4, 5),
20  (8.1, 0.72, 0.0, 1.9, 0.092, 10, 34, 0.9972, 3.18, 0.57, 9.5, 5),
21  (6.2, 0.58, 0.08, 1.9, 0.09, 17, 32, 0.996, 3.27, 0.54, 9.4, 5);
22
```

INFO : Executing state task

INFO : Table default.wine_data stats: [numFiles=2, numRows=23, totalSize=1287, rawDataSize=1264, numFilesErasureCoded=0]

INFO : Completed executing command(queryId=hive_20241108172339_d1364ecd-5a23-4a26-ba29-b46e6e70a670); Time taken: 1.208 seconds

✓ Success.

3. Sorting Data in the Hive Table

 Hive

 

Add a name... Email Survey Opt-Ins, Custom...

    

2.24s default  

```
1 SELECT * FROM wine_data
2 ORDER BY quality ASC;
3
```

Query History Saved Queries Results (23)				
	wine_data.fixed_acidity	wine_data.volatile_acidity	wine_data.citric_acid	wine_data.residual_sugar
1	7.8	0.76	0.04	2.3
2	7.8	0.88	0	2.6
3	7.8	0.76	0.04	2.3
4	7.4	0.7	0	1.9
5	7.8	0.58	0.02	2
6	7.5	0.5	0.36	6.1
7	6.7	0.58	0.08	1.8
8	7.1	0.59	0	1.9
9	7.4	0.7	0	1.9
10	7.8	0.61	0.29	1.6
11	7.5	0.31	0.22	6.8
12	10.5	0.31	0.56	6.2
13	6.8	0.67	0.02	1.8
14	8.1	0.72	0	1.9
15	6.2	0.58	0.08	1.9
16	7.4	0.7	0	1.9
17	7.8	0.88	0	2.6

Sort in Descending Order by Multiple Columns (**alcohol** and **quality**):


Hive
🔄
📄
Add a name...
Email Survey Opt-Ins, Custom...
📈
▼
📁
▼

2.24s default ▼ ⚙️

```

1 SELECT * FROM wine_data
2 ORDER BY alcohol DESC, quality DESC;
3

```

Query History Saved Queries Results (23)

	wine_data.fixed_acidity	wine_data.volatile_acidity	wine_data.citric_acid	wine_data.residual_sugar
1	6.3	0.39	0.16	1.4
2	7.5	0.5	0.36	6.1
3	6.8	0.54	0	2.4
4	7.3	0.65	0	1.2
5	7.3	0.65	0	1.2
6	7.5	0.31	0.22	6.8
7	11.2	0.28	0.56	1.9
8	7.8	0.76	0.04	2.3
9	7.8	0.88	0	2.6
10	7.8	0.76	0.04	2.3
11	7.8	0.88	0	2.6
12	10.5	0.31	0.56	6.2
13	7.8	0.58	0.02	2
14	8.1	0.72	0	1.9
15	7.9	0.6	0.06	1.6
16	7.4	0.7	0	1.9
17	6.8	0.67	0.02	1.8

4. Aggregating Data in the Hive Table

Add a name... Email Survey Opt-Ins, Custom...

4.12s default

```
1 SELECT COUNT(*) AS total_rows FROM wine_data;
```

Query History Saved Queries Results (1)

	total_rows
1	23

Calculate the Sum of a Column (**fixed_acidity**):

Hive     Add a name... Email Survey Opt-Ins, Custom...     

2.25s default  

```
1 SELECT SUM(fixed_acidity) AS total_acidity FROM wine_data;
```

Query History Saved Queries Results (1)

total_acidity

1	176.20000000000002
---	--------------------

  

Find the Average of a Column (**alcohol**):

Hive     Add a name... Email Survey Opt-Ins, Custom...     

0s default  

```
1 SELECT AVG(alcohol) AS avg_alcohol FROM wine_data;  
2
```

Query History   Saved Queries Results (1)

avg_alcohol

1	9.68695652173913
---	------------------

  

5. Combining Sorting and Aggregation

Count Records Grouped by **quality** and Sort by Count in Descending Order:

Hive     Add a name... Email Survey Opt-Ins, Custom...    

2.28s default ▼

```

1 SELECT quality, COUNT(*) AS count_wines
2 FROM wine_data
3 GROUP BY quality
4 ORDER BY count_wines DESC;
5

```

Query History Saved Queries **Results (2)**  

	quality	count_wines
1	5	17
2	6	6

  

Calculate Average **alcohol** Content per **quality** and Sort by Average in Descending Order:

Hive     Add a name... Email Survey Opt-Ins, Custom...     

2.35s default ▼  

```

1 SELECT quality, AVG(alcohol) AS avg_alcohol
2 FROM wine_data
3 GROUP BY quality
4 ORDER BY avg_alcohol DESC;
5

```

Query History Saved Queries **Results (2)**

	quality	avg_alcohol
1	6	9.983333333333334
2	5	9.582352941176472

  

POSTLAB:

1.Explain the difference between Managed and External tables in HiveQL.

Ans:

Managed Table

- Hive manages the table data and metadata.
- Data is stored in a default location within Hive's warehouse directory (e.g., `/user/hive/warehouse`).
- When you drop a managed table, both data and metadata are deleted.
- Suitable for temporary or intermediate data where Hive should manage data lifecycle.

External Table

- Hive manages only the metadata, not the data itself.
- Data can be stored in any specified location, like HDFS or external storage.
- When you drop an external table, only the metadata is deleted; the data remains intact.
- Ideal for shared or persistent data that might be accessed by other applications outside Hive.

2. How do you optimize HiveQL queries?

Ans:

Partitioning:

- Split large tables into partitions based on columns (e.g., date) to reduce the amount of data scanned.

Bucketing:

- Group data within partitions into buckets, which improves performance for joins and sampling.

Use Appropriate File Formats:

- Use columnar formats like ORC or Parquet for better compression and faster reads.

Compression:

- Enable compression for intermediate data to reduce data transfer time.

Avoid **SELECT ***:

- Select only necessary columns to minimize data read and processed.

Use Vectorized Query Execution:

- Enable vectorization for certain operators to process batches of rows, which speeds up operations.

Optimize Joins:

- Use map-side joins for small tables and sort-merge joins for large tables.

Increase Parallelism:

- Adjust configuration to increase the number of mappers and reducers for better parallel processing.